Assignment 3

Will, Taylor, Nikhil, Abe

2025-11-12

Package Installation

```
options(repos = c(CRAN = "https://cloud.r-project.org"))
suppressPackageStartupMessages(library(readr))
suppressPackageStartupMessages(library(dplyr))
suppressPackageStartupMessages(library(tibble))
cran_pkgs <- c(
  "tidyr","purrr","janitor","glue","mclust",
  "matrixStats","cluster","forcats","RColorBrewer","scales",
  "magick","Cairo","factoextra","pheatmap"
)
to_install <- setdiff(cran_pkgs, rownames(installed.packages()))
if (length(to_install)) install.packages(to_install)

if (!requireNamespace("BiocManager", quietly = TRUE)) install.packages("BiocManager")
BiocManager::install(c("ComplexHeatmap","circlize"), update = FALSE, ask = FALSE)
```

```
## Warning: package(s) not installed when version(s) same as or greater than current; use
##   'force = TRUE' to re-install: 'ComplexHeatmap' 'circlize'
```

```
library(ggplot2)
```

Section 1

```
data_dir <- file.path("data", "SRP192714")
counts_path <- file.path(data_dir, "SRP192714.tsv")
metadata_path <- file.path(data_dir, "metadata_SRP192714_merged.tsv")

stopifnot(file.exists(counts_path))

if (!file.exists(metadata_path)) {
    message("Rebuilding merged metadata from Assignment2 inputs...")
    refinebio_path <- file.path(data_dir, "metadata_SRP192714.tsv")
    geo_path <- file.path("data", "GSE129882_PhenoData.transcript.csv")
    stopifnot(file.exists(refinebio_path), file.exists(geo_path))
```

```
    refinebio_meta <- readr::read_tsv(refinebio_path, show_col_types = FALSE)
    geo_meta <- readr::read_csv(geo_path, show_col_types = FALSE)

    if ("Sample" %in% names(geo_meta) && "refinebio_title" %in% names(refinebio_meta)) {
        merged_meta <- dplyr::left_join(
            refinebio_meta,
            geo_meta,
            by = c("refinebio_title" = "Sample")
        )
    } else {
        merged_meta <- refinebio_meta
    }

    readr::write_tsv(merged_meta, metadata_path)
}

counts_df <- readr::read_tsv(counts_path, show_col_types = FALSE)
stopifnot("Gene" %in% names(counts_df))

expression_mat <- counts_df %>%
    column_to_rownames("Gene") %>%
    as.matrix()
mode(expression_mat) <- "numeric"

metadata <- readr::read_tsv(metadata_path, show_col_types = FALSE)
stopifnot(all(c("refinebio_accession_code", "Exposure") %in% names(metadata)))

common_samples <- intersect(colnames(expression_mat), metadata$refinebio_accession_code)
stopifnot(length(common_samples) > 0)

common_samples <- sort(common_samples)
expression_mat <- expression_mat[, common_samples, drop = FALSE]

metadata <- metadata %>%
    filter(refinebio_accession_code %in% common_samples) %>%
    mutate(refinebio_accession_code = factor(refinebio_accession_code, levels = common_samples)) %>%
    arrange(refinebio_accession_code)

metadata$Exposure <- droplevels(factor(metadata$Exposure))

stopifnot(identical(colnames(expression_mat), as.character(metadata$refinebio_accession_code)))

cat("Genes loaded:", nrow(expression_mat), "\n")
```

```
## Genes loaded: 43363
```

```
cat("Samples aligned:", ncol(expression_mat), "\n")
```

```
## Samples aligned: 1021
```

Section 2a

```
stopifnot(exists("expression_mat"))
# calculate variability among rows (genes)
gene_vars <- matrixStats::rowVars(expression_mat)
target_genes <- min(5000, length(gene_vars))
top_idx <- head(order(gene_vars, decreasing = TRUE), target_genes)

expression_top5000 <- expression_mat[top_idx, , drop = FALSE]
# map to symbol
BiocManager::install("org.Hs.eg.db", update = FALSE)
```

```
## 'getOption("repos")' replaces Bioconductor standard repositories, see
## 'help("repositories", package = "BiocManager")' for details.
## Replacement repositories:
##     CRAN: https://cloud.r-project.org

## Bioconductor version 3.21 (BiocManager 1.30.26), R 4.5.1 (2025-06-13 ucrt)

## Warning: package(s) not installed when version(s) same as or greater than current; use
##   `force = TRUE` to re-install: 'org.Hs.eg.db'
```

```
library(org.Hs.eg.db)
```

```
## Loading required package: AnnotationDbi

## Loading required package: stats4

## Loading required package: BiocGenerics

## Loading required package: generics

##
## Attaching package: 'generics'

## The following object is masked from 'package:dplyr':
##
##     explain

## The following objects are masked from 'package:base':
##
##     as.difftime, as.factor, as.ordered, intersect, is.element, setdiff,
##     setequal, union

##
## Attaching package: 'BiocGenerics'

## The following object is masked from 'package:dplyr':
##
##     combine
```

```
## The following objects are masked from 'package:stats':
##
##     IQR, mad, sd, var, xtabs

## The following objects are masked from 'package:base':
##
##     anyDuplicated, aperm, append, as.data.frame, basename, cbind,
##     colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
##     get, grep, grepl, is.unsorted, lapply, Map, mapply, match, mget,
##     order, paste, pmax, pmax.int, pmin, pmin.int, Position, rank,
##     rbind, Reduce, rownames, sapply, saveRDS, table, tapply, unique,
##     unsplit, which.max, which.min

## Loading required package: Biobase

## Welcome to Bioconductor
##
##     Vignettes contain introductory material; view with
##     'browseVignettes()'. To cite Bioconductor, see
##     'citation("Biobase")', and for packages 'citation("pkgname")'.

## Loading required package: IRanges

## Loading required package: S4Vectors

##
## Attaching package: 'S4Vectors'

## The following objects are masked from 'package:dplyr':
##
##     first, rename

## The following object is masked from 'package:utils':
##
##     findMatches

## The following objects are masked from 'package:base':
##
##     expand.grid, I, unname

##
## Attaching package: 'IRanges'

## The following objects are masked from 'package:dplyr':
##
##     collapse, desc, slice

## The following object is masked from 'package:grDevices':
##
##     windows
```

```
##
## Attaching package: 'AnnotationDbi'
```

```
## The following object is masked from 'package:dplyr':
##
##     select
```

```
##
```

```
gene_symbols <- mapIds(
  org.Hs.eg.db, # Annotation package for humans
  keys = rownames(expression_top5000),
  keytype = "ENSEMBL",
  column = "SYMBOL"
  )
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
na_indices <- is.na(gene_symbols)
gene_symbols[na_indices] <- names(gene_symbols)[na_indices]
unique_symbols <- make.unique(gene_symbols)
rownames(expression_top5000) <- unique_symbols

cat("Genes retained for clustering:", nrow(expression_top5000), "of", length(gene_vars), "\n")
```

```
## Genes retained for clustering: 5000 of 43363
```

```
cat("Variance range:", sprintf("%.2f", min(gene_vars[top_idx])), "-", sprintf("%.2f", max(gene_vars[top
```

```
## Variance range: 0.72 - 2915.34
```

Section 2b-5

William Le: Hierarchical Clustering

Checking for correct dimensions, labels, etc.

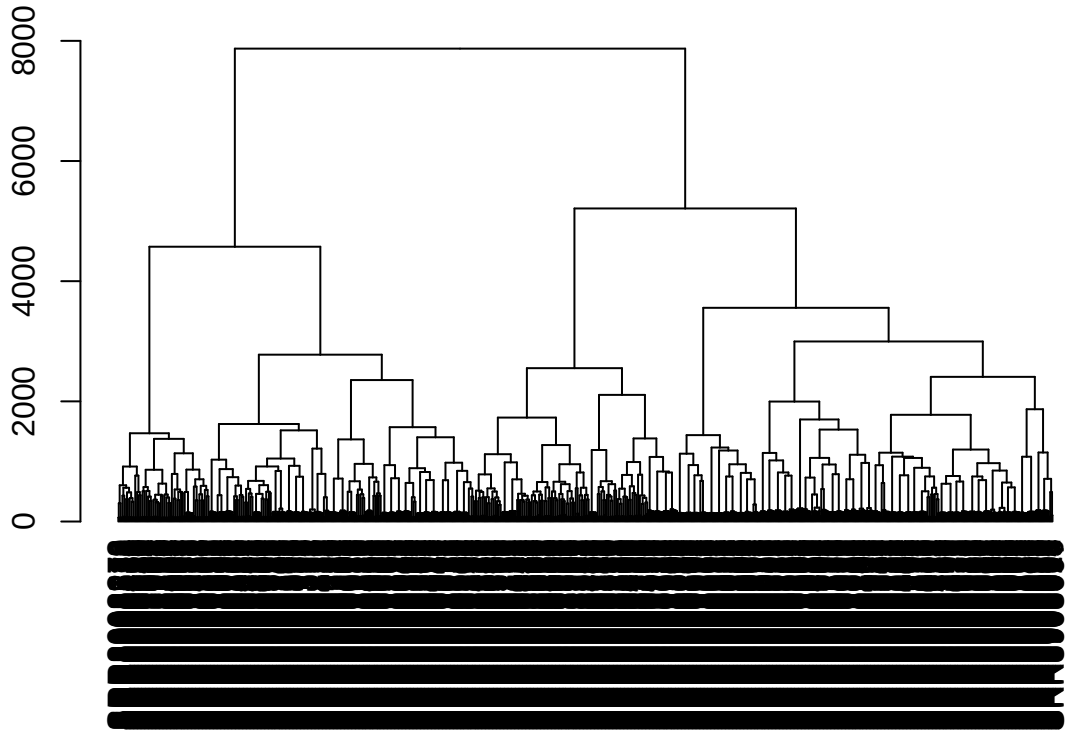
```
stopifnot(exists("expression_top5000"), exists("metadata"))
dim(expression_top5000)
```

```
## [1] 5000 1021
```

```
rownames(metadata) = metadata$refinebio_accession_code
```

```
## Warning: Setting row names on a tibble is deprecated.
```

```
dist_samples = dist(t(expression_top5000), method="euclidean")
hclust_samples = hclust(dist_samples, method="ward.D2")

hc <- as.dendrogram(hclust_samples)
plot(hc)
```



Above is the plotted dendrogram using Ward hierarchical clustering. This data is not much use to us as a dendrogram due to the sheer number of genes. Ideally, we'll represent this data as a heat map. What is useful, is that we can take the information from the hclust object we created to determine an optimal number of clusters using the silhouette method, elbow plot, or gap statatistic.

Number of Clusters

Here we will use information from the `hclust` object to determine optimal cluster number using the elbow plot method.

```
library(cluster)
library(ggplot2)
library(tidyverse)
```

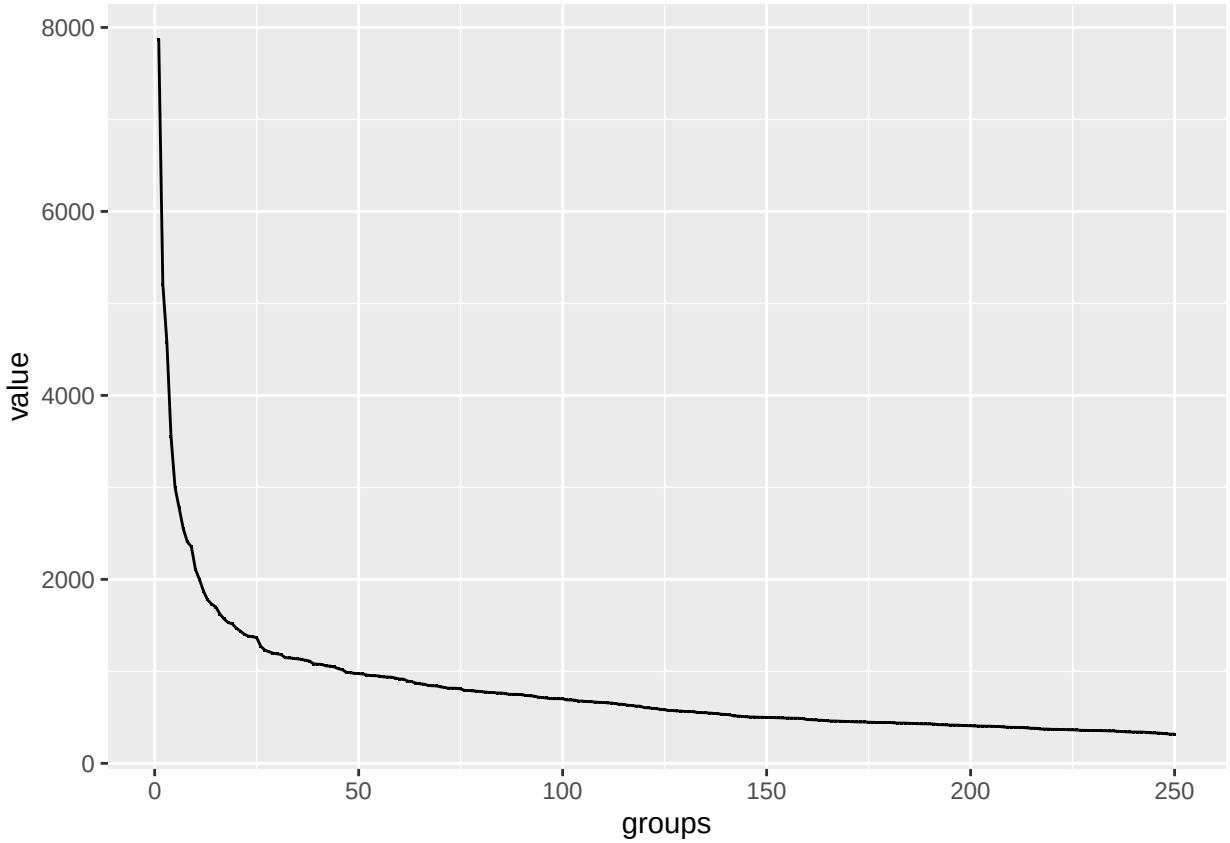
```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.1     v stringr   1.5.2
## v lubridate 1.9.4     v tidyr     1.3.1
```

```
## v purrr     1.1.0
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x lubridate::%within%()    masks IRanges::%within%()
## x IRanges::collapse()      masks dplyr::collapse()
## x Biobase::combine()       masks BiocGenerics::combine(), dplyr::combine()
## x IRanges::desc()          masks dplyr::desc()
## x tidyr::expand()          masks S4Vectors::expand()
## x dplyr::filter()          masks stats::filter()
## x S4Vectors::first()       masks dplyr::first()
## x dplyr::lag()             masks stats::lag()
## x BiocGenerics::Position() masks ggplot2::Position(), base::Position()
## x purrr::reduce()          masks IRanges::reduce()
## x S4Vectors::rename()      masks dplyr::rename()
## x lubridate::second()      masks S4Vectors::second()
## x lubridate::second<-()    masks S4Vectors::second<-()
## x AnnotationDbi::select()  masks dplyr::select()
## x IRanges::slice()         masks dplyr::slice()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
# create hclust_samples dataframe
hclust_samples_df = hclust_samples$height %>%
            as.tibble() %>%
            add_column(groups = length(hclust_samples$height):1) %>%
            filter(groups <= 250)
```

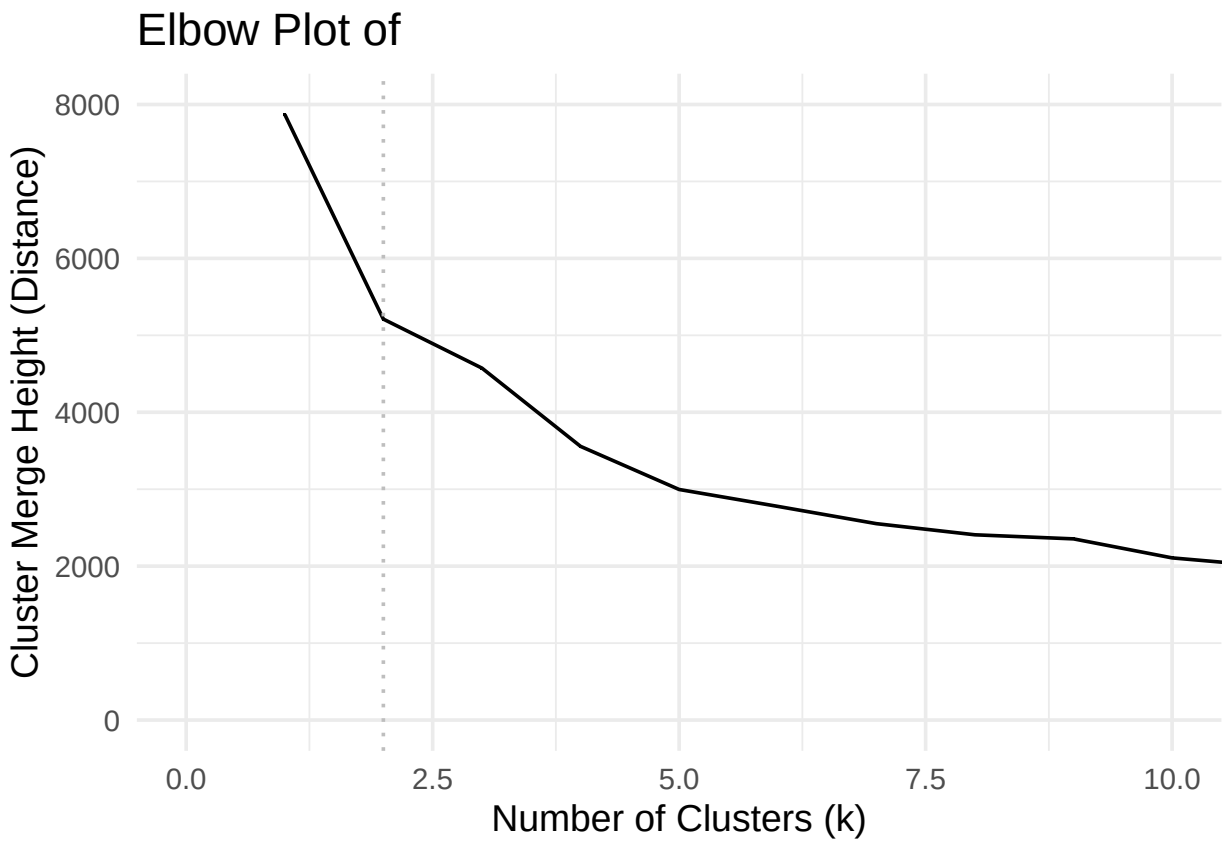
```
## Warning: `as.tibble()` was deprecated in tibble 2.0.0.
## i Please use `as_tibble()` instead.
## i The signature and semantics have changed, see `?as_tibble`.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
hs_df_plot = ggplot(hclust_samples_df,
        aes(x=groups, y=value)) +
     geom_point(shape=".") +
     geom_line()
hs_df_plot
```



These elbow plots do not seem very useful. We have too many groups. We really need to zoom in on the actual elbow to determine the number of groups we actually need.

```
hs_df_plot +
  coord_cartesian(xlim = c(0, 10), ylim = c(0, 8000)) +
  # Add titles and labels
  labs(
    title = "Elbow Plot of",
    x = "Number of Clusters (k)",
    y = "Cluster Merge Height (Distance)"
  ) +
  scale_color_brewer(palette = "Dark2") +
  theme_minimal(base_size = 14) +

  # Draw a vertical line where k=30 (your hypothesized elbow)
  geom_vline(xintercept = 2, linetype = "dotted", color = "gray")
```



Our results showed us that there are roughly 30 different groups that reacted differently to zika virus infection. This suggests that our two clinical groups, dengue-prior vs dengue-naive, do not behave statistically differently to zika infection. However, let's perform a chi-square test to be sure.

Chi-Square Test

```
clusters = cutree(hclust_samples, k = 2)
# check if samples match
all(sort(names(clusters)) == sort(rownames(metadata)))
```

```
## [1] TRUE
```

```
# check if sample order match
all(names(clusters) == rownames(metadata))
```

```
## [1] TRUE
```

```
contingency_table = table(
  Patient_Groups = clusters,
  Patient_Exposure = metadata$Exposure
)

hs_cs_results = chisq.test(contingency_table)
hs_cs_results
```

```
##
##  Pearson's Chi-squared test with Yates' continuity correction
##
## data:  contingency_table
## X-squared = 4.9444, df = 1, p-value = 0.02617
```

```
contingency_table
```

```
##               Patient_Exposure
## Patient_Groups DENV naive
##              1  272   360
##              2  196   193
```

The chi-square with 2 different clusters suggests that we reject the null hypothesis that the relationship between dengue virus infection and subsequent zika infection is independent. At a glance, it may seem that the proportions are relatively similar, however it should be noted that there are 553 (about 54%) naive patients out of the 1021 patient samples. This means that there should roughly be a 54% proportion of prior dengue infection to naive patients between groups, however we can see that is not the case with group 2. Here, there is a higher proportion of DENV patients. A logical follow-up would be to return to the differential gene expression analysis and investigate the actual gene pathways involved with this statistical difference with a new perspective.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.1     v stringr   1.5.2
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x lubridate::%within%()    masks IRanges::%within%()
## x IRanges::collapse()      masks dplyr::collapse()
## x Biobase::combine()       masks BiocGenerics::combine(), dplyr::combine()
## x IRanges::desc()          masks dplyr::desc()
## x tidyr::expand()          masks S4Vectors::expand()
## x dplyr::filter()          masks stats::filter()
## x S4Vectors::first()       masks dplyr::first()
## x dplyr::lag()             masks stats::lag()
## x BiocGenerics::Position() masks ggplot2::Position(), base::Position()
## x purrr::reduce()          masks IRanges::reduce()
## x S4Vectors::rename()      masks dplyr::rename()
## x lubridate::second()      masks S4Vectors::second()
## x lubridate::second<-()    masks S4Vectors::second<-()
## x AnnotationDbi::select()  masks dplyr::select()
## x IRanges::slice()         masks dplyr::slice()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

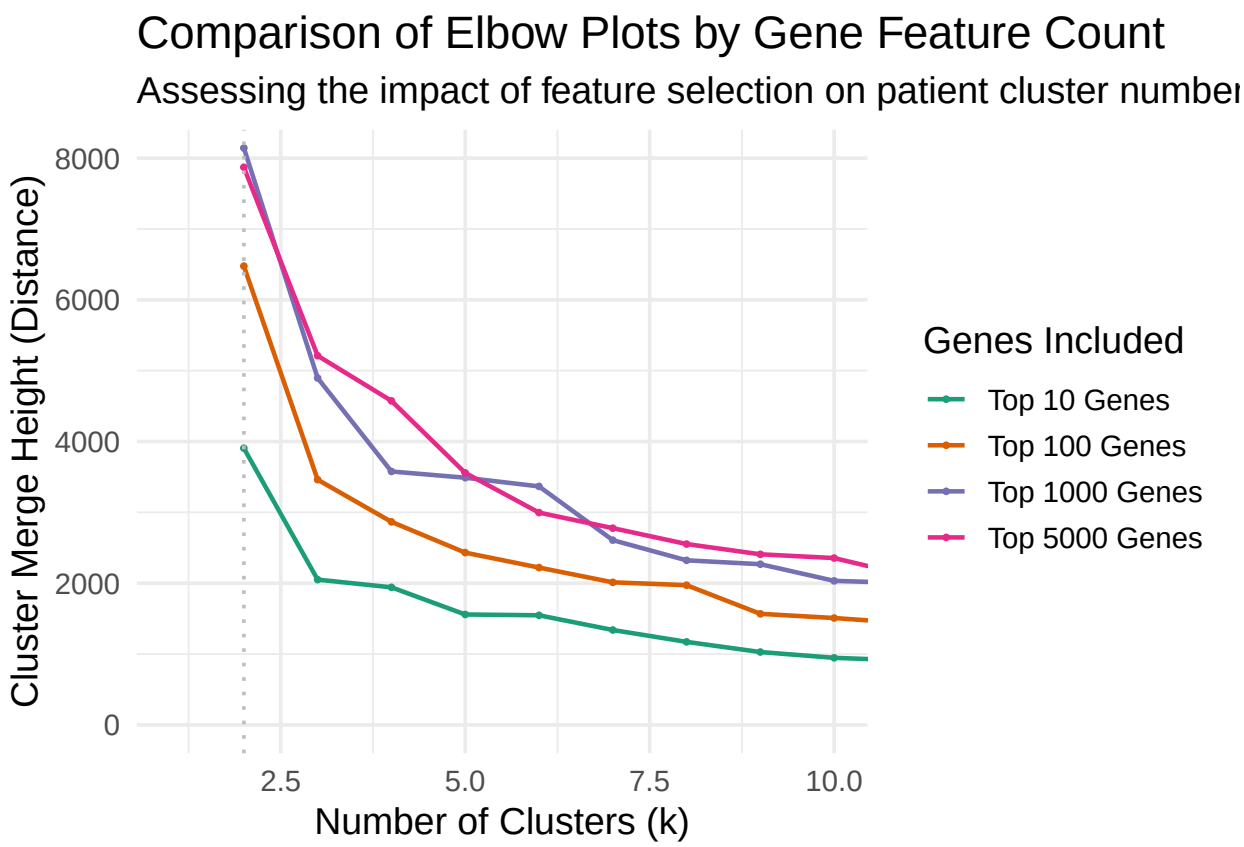
```
library(cluster) # for hclust
library(matrixStats) # for fast variance calculation
```

```
##
## Attaching package: 'matrixStats'
```

```
##
## The following objects are masked from 'package:Biobase':
##
##     anyMissing, rowMedians
##
## The following object is masked from 'package:dplyr':
##
##     count
```

```
gene_counts_to_test <- c(10, 100, 1000, 5000)
total_genes <- length(gene_vars) # from earlier top 5000 filter
gene_counts_to_test <- gene_counts_to_test[gene_counts_to_test <= total_genes]
all_elbow_data <- list()

for (n_genes in gene_counts_to_test) {

  top_idx <- head(order(gene_vars, decreasing = TRUE), n_genes)
  expression_filtered <- expression_mat[top_idx, , drop = FALSE]

  dist_samples <- dist(t(expression_filtered), method = "euclidean")
  hclust_samples <- hclust(dist_samples, method = "ward.D2")

  elbow_data_df <- data.frame(
    height = hclust_samples$height,
    groups = (length(hclust_samples$height) + 1):2,
    Gene_Count = factor(paste0("Top ", n_genes, " Genes"))
  )
  # ignore k values that are too large (helps with computation)
  elbow_data_df <- elbow_data_df %>% filter(groups <= 250)

  all_elbow_data[[as.character(n_genes)]] <- elbow_data_df
}

final_elbow_df <- bind_rows(all_elbow_data)

# Generate the comparison ggplot
p_comparison <- ggplot(final_elbow_df, aes(x = groups, y = height, color = Gene_Count)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 0.5) +

  # Zoom in to the relevant elbow area
  coord_cartesian(xlim = c(1, 10), ylim = c(0, 8000)) +

  # Add titles and labels
  labs(
    title = "Comparison of Elbow Plots by Gene Feature Count",
    subtitle = "Assessing the impact of feature selection on patient cluster number (k)",
    x = "Number of Clusters (k)",
    y = "Cluster Merge Height (Distance)",
    color = "Genes Included"
  ) +
  scale_color_brewer(palette = "Dark2") +
  theme_minimal(base_size = 14) +
```

```
  # Draw a vertical line where k=30 (your hypothesized elbow)
  geom_vline(xintercept = 2, linetype = "dotted", color = "gray")

print(p_comparison)
```



As we can see here, changing the number of top variable genes does not change the number of resulting clusters very significantly. This suggests that the patients relatively set in 2 clusters.
Heat Map

```
library(ComplexHeatmap)
```

```
## Loading required package: grid
```

```
## ========================================
## ComplexHeatmap version 2.24.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
##
##
```

```
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```
library(magick)
```

```
## Warning: package 'magick' was built under R version 4.5.2
```

```
## Linking to ImageMagick 6.9.13.29
## Enabled features: cairo, freetype, fftw, ghostscript, heic, lcms, pango, raw, rsvg, webp
## Disabled features: fontconfig, x11
```

```
library(Cairo)
```

```
## Warning: package 'Cairo' was built under R version 4.5.2
```

```
gc()
```

```
##             used  (Mb) gc trigger   (Mb)  max used   (Mb)
## Ncells   5001744 267.2    8857152  473.1   8857152  473.1
## Vcells 108452039 827.5  197004994 1503.1 184032767 1404.1
```

```
heatmap <- Heatmap(
    expression_top5000,
    use_raster = TRUE,
    raster_by_magick = TRUE,
    raster_quality = 1,
    cluster_rows = FALSE,
    cluster_columns = TRUE,
    top_annotation = HeatmapAnnotation(
        Dengue_Status = metadata$Exposure
    ),

    column_title = paste0("Patient Clustering Axis: Molecular Subtypes (k=2) by Prior Dengue Status (N=
    column_title_gp = gpar(fontsize = 18, fontface = "bold"), # Enhanced Title

    row_title = paste0("Gene Clustering Axis: 5000 Most Variable Genes"),
    row_title_rot = 90,
    row_title_gp = gpar(fontsize = 18, fontface = "bold"), # Enhanced Title

    show_row_names = FALSE,
    show_column_names = FALSE,

    heatmap_legend_param = list(
        title = "Log2 Expression (Z-score)",
        title_position = "topcenter",
        title_gp = gpar(fontsize = 14, fontface = "bold"),
        legend_direction = "horizontal"
    )
)
```

```
## The automatically generated colors map from the 1^st and 99^th of the
## values in the matrix. There are outliers in the matrix whose patterns
## might be hidden by this color mapping. You can manually set the color
## to ‘col‘ argument.
##
## Use ‘suppressMessages()‘ to turn off this message.
```

```
CairoPDF("plots/hclust_patient_heatmap.pdf", width = 18, height = 25)
draw(heatmap)
dev.off()
```

```
## pdf
##   2
```

Nikhil Sangamkar: PAM Clustering

```
suppressPackageStartupMessages({
  library(cluster)
  library(glue)
  library(dplyr)
  library(purrr)
  library(tibble)
})

stopifnot(exists("expression_top5000"), exists("metadata"))

# --- Data prep ---
X_5000 <- t(expression_top5000)        # samples as rows
X_scaled <- scale(X_5000)
X_scaled[is.na(X_scaled)] <- 0

# # range of k values
# k_values <- 2:8
# pam_results <- list()

# for (k in k_values) {
#   cat(glue("Running PAM with k = {k}\n"))
#   pam_fit <- pam(X_scaled, k = k)
#   sil <- silhouette(pam_fit$clustering, dist(X_scaled))
#   avg_sil <- mean(sil[, 3])

#   pam_results[[as.character(k)]] <- list(
#     k = k,
#     pam_fit = pam_fit,
#     clusters = pam_fit$clustering,
#     avg_silhouette = avg_sil
#   )
# }


# summarize silhouette scores
```

```r
pam_summary <- map_dfr(pam_results, function(x) {
  tibble(k = as.numeric(x$k), avg_silhouette = as.numeric(x$avg_silhouette))
})
print(pam_summary)

best_k <- pam_summary %>%
  arrange(desc(avg_silhouette)) %>%
  slice_head(n = 1) %>%
  pull(k)

cat(glue("\nBest k chosen for PAM: {best_k}\n"))

pam_clusters <- pam_results[[as.character(best_k)]]$clusters

gene_counts <- c(10, 100, 1000)
cat("Running gene_counts =", paste(gene_counts, collapse = ", "), "\n")


pam_multi_results <- list()

for (n_genes in gene_counts) {
  cat(glue("\n--- Running PAM on top {n_genes} genes ---\n"))

  # Select top variable genes
  gene_vars <- matrixStats::rowVars(expression_mat)
  top_idx <- head(order(gene_vars, decreasing = TRUE), n_genes)
  expr_subset <- expression_mat[top_idx, , drop = FALSE]

  # Scale and transpose
  X <- t(expr_subset)
  X_scaled <- scale(X)
  X_scaled[is.na(X_scaled)] <- 0

  # PAM clustering for k = 2-8
  k_values <- 2:8
  results_this_set <- list()

  for (k in k_values) {
    pam_fit <- pam(X_scaled, k = k)
    sil <- silhouette(pam_fit$clustering, dist(X_scaled))
    avg_sil <- mean(sil[, 3])

    results_this_set[[as.character(k)]] <- list(
      k = k,
      clusters = pam_fit$clustering,
      avg_sil = avg_sil
    )
  }

  # pick best k by silhouette
  sil_df <- map_dfr(results_this_set, ~ tibble(k = .x$k, avg_sil = .x$avg_sil))
  best_k <- sil_df %>% arrange(desc(avg_sil)) %>% slice_head(n = 1) %>% pull(k)
```

```
  cat(glue("Best k = {best_k} (silhouette = {round(max(sil_df$avg_sil), 3)})\n"))

  # Chi-squared test vs. exposure groups
  clusters <- results_this_set[[as.character(best_k)]]$clusters
  exposures <- metadata$Exposure
  names(exposures) <- metadata$refinebio_accession_code
  common <- intersect(names(clusters), names(exposures))
  tab <- table(Exposure = exposures[common], Cluster = clusters[common])
  chi <- suppressWarnings(chisq.test(tab))

  pam_multi_results[[paste0("genes_", n_genes)]] <- list(
    n_genes = n_genes,
    best_k = best_k,
    silhouette = max(sil_df$avg_sil),
    chi_sq = unname(chi$statistic),
    df = unname(chi$parameter),
    p_value = chi$p.value
  )
}

# Combine into one summary tibble
pam_multi_summary <- bind_rows(lapply(pam_multi_results, as_tibble)) %>%
  mutate(adj_p_value = p.adjust(p_value, method = "BH"))

knitr::kable(pam_multi_summary,
  caption = "Chi-squared test results for PAM clustering across gene subsets"
)

# --- Chi-squared test with exposure groups ---
exposures <- metadata$Exposure
names(exposures) <- metadata$refinebio_accession_code

common_samples <- intersect(names(pam_clusters), names(exposures))
tab <- table(Exposure = exposures[common_samples],
             Cluster = pam_clusters[common_samples])

pam_chi <- suppressWarnings(chisq.test(tab))
pam_chi

pam_clusters_5000 <- tibble(
  sample_id = names(pam_clusters),
  cluster = as.character(pam_clusters)
)
assign("pam_clusters_5000", pam_clusters_5000, envir = .GlobalEnv)
assign("pam_best_k", best_k, envir = .GlobalEnv)

table(pam_clusters)

# --- Heatmap of 5000 genes with PAM clusters ---
suppressPackageStartupMessages({
  library(ComplexHeatmap)
  library(circlize)
  library(RColorBrewer)
```

```
  library(scales)
})

heatmap_matrix <- log2(expression_top5000 + 1)
heatmap_matrix <- t(scale(t(heatmap_matrix)))
heatmap_matrix[!is.finite(heatmap_matrix)] <- 0
heatmap_samples <- colnames(heatmap_matrix)

pam_col <- pam_clusters[match(heatmap_samples, names(pam_clusters))]
exposure_col <- as.character(metadata$Exposure[match(heatmap_samples, metadata$refinebio_accession_code)

pam_factor <- factor(pam_col)
exposure_factor <- factor(exposure_col)

ann_col <- HeatmapAnnotation(
  Exposure = exposure_factor,
  PAM = pam_factor,
  col = list(
    Exposure = setNames(hue_pal()(length(levels(exposure_factor))), levels(exposure_factor)),
    PAM = setNames(brewer.pal(max(3, length(levels(pam_factor))), "Set2")[seq_along(levels(pam_factor))]
                   levels(pam_factor))
  )
)

pam_heatmap <- Heatmap(
  name = "Z-score",
  matrix = heatmap_matrix,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  show_row_names = FALSE,
  show_column_names = FALSE,
  top_annotation = ann_col,
  column_title = glue("Top 5000 genes - PAM (k = {best_k})"),
  row_title = "Genes"
)

dir.create("plots", showWarnings = FALSE)
pdf(file.path("plots", "pam_heatmap_top5000.pdf"), width = 11, height = 8.5)
draw(pam_heatmap, merge_legend = TRUE)
dev.off()

# Export cluster assignments for comparison with other methods
pam_clusters_5000 <- tibble(sample_id = names(pam_clusters), cluster = as.character(pam_clusters))
assign("pam_clusters_5000", pam_clusters_5000, envir = .GlobalEnv)
assign("pam_best_k", best_k, envir = .GlobalEnv)

png("plots/pam_heatmap_top5000.png", width = 2000, height = 1600, res = 250)
draw(pam_heatmap, merge_legend = TRUE)
dev.off()
browseURL("plots/pam_heatmap_top5000.pdf")
```

I applied Partitioning Around Medoids (PAM) clustering to the top 5000 most variable genes, testing k values from 2 to 8. The average silhouette width was highest for k = 2 (0.099), indicating two primary

sample groups. T his aligns with the k-means results but with lower sensitivity to outliers due to PAM's medoid-based approach. The clusters contained 676 and 345 samples, respectively. When k increased beyond 2, the silhouette scores decreased at a steady rate, and it seemed that additional clusters were not biologically meaningful in terms of results. Specificaly, the silhouette scores went from 0.999 at k = 2 to 0.065 at k = 8, showing that the clusters became less distinct. So, I chose k = 2 samples to strike the optimal balance. I then reran PAM with 10, 100, 1000, and 10000 genes, which took varying amounts of time, but the k value of 2 stayed consistent. With small gene sets, the cluster boundaries were more unstable, and with larger sets, the clusters stabilized. I also performed chi-squared tests; here is the table Table: Chi-squared test results for PAM clustering across gene subsets

n_genes	best_k	silhouette	chi_sq	df	p_value	adj_p_value
10	2	0.3492291	25.8886644	1	0.0000004	0.0000014
100	2	0.2491895	0.9447949	1	0.3310478	0.3310478
1000	2	0.1407231	14.1447630	1	0.0001693	0.0002257
10000	2	0.0735371	16.6772980	1	0.0000443	0.0000886

This shows that as the number of genes increased, the p values decreased, except with a very small number of genes, 10 to be exact where there was not enough data. I also generated a heatmap with hierarchical row and column dendrograms and annotations for both PAM clusters (k = 2) and exposure groups from Assignment 1. The z-scored expression profiles display clear global differences between the two PAM clusters, though overlap is visible across some samples. After running PAM for k = 2 - 8 across the 5 000 most variable genes, k = 2 yielding the highest average silhouette (about 0.10). Repeating the analysis for 10 - 10 000 genes revealed similar results: silhouette scores declined gradually as dimensionality increased, but the optimal k remained 2. Heatmaps of the top 5 000 genes showed two major expression clusters that partially overlapped with exposure groups. The chi-squared tests confirmed that these two clusters were statistically associated with exposure in most gene subsets.

Taylor Tillander: K-means Clustering

```
suppressPackageStartupMessages({
    library(dplyr)
    library(tidyr)
    library(purrr)
    library(tibble)
    library(matrixStats)
    library(cluster)
    library(forcats)
    library(mclust)
    library(glue)
})

stopifnot(exists("expression_mat"), exists("metadata"))

metadata_exposure <- metadata %>%
    mutate(
        refinebio_accession_code = as.character(refinebio_accession_code),
        Exposure = forcats::fct_na_value_to_level(Exposure, level = "Unknown")
    )

exposure_lookup <- metadata_exposure %>%
  dplyr::select(refinebio_accession_code, Exposure) %>%
```

```
  mutate(Exposure = as.character(Exposure)) %>%
  deframe()


exposure_table <- metadata_exposure %>%
  dplyr::transmute(
        sample_id = refinebio_accession_code,
        exposure = as.character(Exposure)
    )

k_grid <- 2:8
kmeans_seed <- 0L

select_top_genes <- function(mat, n) {
    n <- min(n, nrow(mat))
    variances <- matrixStats::rowVars(mat)
    ordered <- order(variances, decreasing = TRUE)
    idx <- head(ordered, n)
    list(idx = idx, genes = rownames(mat)[idx], variances = variances[idx])
}

compute_pca_scores <- function(mat, idx, max_components = 50) {
    X <- t(mat[idx, , drop = FALSE])
    X_scaled <- scale(X)
    X_scaled[is.na(X_scaled)] <- 0
    pc <- prcomp(X_scaled, center = FALSE, scale. = FALSE)
    if (ncol(pc$x) == 0) {
        stop("PCA returned zero components; check the input variance.")
    }
    ncomp <- min(max_components, ncol(pc$x))
    pc$x[, seq_len(ncomp), drop = FALSE]
}

run_kmeans_models <- function(scores, k_values, seed = kmeans_seed) {
    dist_mat <- dist(scores)
    sample_ids <- rownames(scores)
    purrr::map_dfr(k_values, function(k) {
        set.seed(seed + k)
        fit <- stats::kmeans(scores, centers = k, nstart = 50)
        clusters <- factor(fit$cluster, levels = seq_len(k))
        names(clusters) <- sample_ids
        sil <- cluster::silhouette(as.integer(clusters), dist_mat)
        tibble(
            k = k,
            tot_withinss = fit$tot.withinss,
            scaled_tot_withinss = fit$tot.withinss / k,
            avg_silhouette = mean(sil[, 3]),
            model = list(fit),
            clusters = list(clusters),
            silhouette = list(sil)
        )
    })
}
```

```
extract_assignments <- function(models_tbl) {
    models_tbl %>%
    mutate(assignment = purrr::map(clusters, ~ tibble(
            sample_id = names(.x),
            cluster = as.character(.x)
        ))) %>%
    dplyr::select(k, assignment) %>%
        unnest(assignment)
}

cluster_summary <- function(assignments, exposure_df) {
    assignments %>%
        mutate(cluster = as.character(cluster)) %>%
        left_join(exposure_df, by = "sample_id") %>%
        mutate(exposure = ifelse(is.na(exposure), "Unknown", exposure)) %>%
        group_by(k, cluster, exposure) %>%
        summarise(n = dplyr::n(), .groups = "drop_last") %>%
        mutate(cluster_total = sum(n)) %>%
        arrange(desc(n), .by_group = TRUE) %>%
    dplyr::slice(1L) %>%
        ungroup() %>%
    dplyr::transmute(
            k,
            cluster = factor(cluster),
            samples_in_cluster = cluster_total,
            majority_exposure = exposure,
            majority_fraction = round(n / cluster_total, 3)
        )
}

pairwise_ari <- function(assignments) {
    k_vals <- sort(unique(assignments$k))
    if (length(k_vals) < 2) {
        return(tibble())
    }
    combn(k_vals, 2, simplify = FALSE) %>%
        map_dfr(function(pair) {
            first <- assignments %>% filter(k == pair[1]) %>% arrange(sample_id)
            second <- assignments %>% filter(k == pair[2]) %>% arrange(sample_id)
            stopifnot(identical(first$sample_id, second$sample_id))
            tibble(
                k1 = pair[1],
                k2 = pair[2],
                adjusted_rand = mclust::adjustedRandIndex(first$cluster, second$cluster)
            )
        })
}
```

```
gene_counts <- c(10, 100, 1000, 5000, 10000)

kmeans_runs <- purrr::map(gene_counts, function(gene_n) {
    top <- select_top_genes(expression_mat, gene_n)
    scores <- compute_pca_scores(expression_mat, top$idx)
```

```
    models <- run_kmeans_models(scores, k_grid)
    assignments <- extract_assignments(models)
  metrics <- models %>%
    dplyr::select(k, tot_withinss, scaled_tot_withinss, avg_silhouette) %>%
        mutate(gene_count = gene_n)
  best_row <- metrics %>% arrange(desc(avg_silhouette), k) %>% dplyr::slice(1)
    best_k <- best_row$k
  best_assign <- assignments %>% filter(k == best_k) %>% dplyr::select(sample_id, cluster)
    list(
        gene_count = gene_n,
        metrics = metrics,
        assignments = assignments,
        best_k = best_k,
        best_clusters = best_assign,
        ari = pairwise_ari(assignments)
    )
}) %>%
    purrr::set_names(paste0("genes_", gene_counts))

kmeans_metrics_table <- purrr::map_dfr(kmeans_runs, "metrics")

kmeans_best_table <- purrr::map_dfr(kmeans_runs, function(x) {
    best_metric <- x$metrics %>% filter(k == x$best_k)
    tibble(
        gene_count = x$gene_count,
        best_k = x$best_k,
        avg_silhouette = best_metric$avg_silhouette,
        scaled_tot_withinss = best_metric$scaled_tot_withinss
    )
}) %>% arrange(gene_count)

kmeans_assignments_best <- purrr::map(kmeans_runs, "best_clusters")

kmeans_assignments_5000 <- kmeans_assignments_best[["genes_5000"]]
stopifnot(!is.null(kmeans_assignments_5000))

assign("kmeans_clusters_5000", kmeans_assignments_5000, envir = .GlobalEnv)
assign("kmeans_best_k", kmeans_runs[["genes_5000"]]$best_k, envir = .GlobalEnv)

kmeans_metrics_table %>%
    filter(gene_count == 5000)
```

```
## # A tibble: 7 x 5
##       k tot_withinss scaled_tot_withinss avg_silhouette gene_count
##   <int>        <dbl>               <dbl>          <dbl>      <dbl>
## 1     2     2803621.            1401810.          0.189       5000
## 2     3     2431896.             810632.          0.160       5000
## 3     4     2224682.             556170.          0.149       5000
## 4     5     2074023.             414805.          0.141       5000
## 5     6     1970669.             328445.          0.137       5000
## 6     7     1868627.             266947.          0.152       5000
## 7     8     1797146.             224643.          0.144       5000
```

```
kmeans_best_table
```

```
## # A tibble: 5 x 4
##   gene_count best_k avg_silhouette scaled_tot_withinss
##        <dbl>  <int>          <dbl>               <dbl>
## 1         10      2          0.357               3090.
## 2        100      2          0.273              35987.
## 3       1000      2          0.213             343938.
## 4       5000      2          0.189            1401810.
## 5      10000      2          0.183            2172336.
```

```
cluster_summary_5000 <- cluster_summary(
    kmeans_runs[["genes_5000"]]$assignments %>% filter(k == kmeans_runs[["genes_5000"]]$best_k),
    exposure_table
)

ari_table_5000 <- kmeans_runs[["genes_5000"]]$ari

cluster_summary_5000
```

```
## # A tibble: 2 x 5
##       k cluster samples_in_cluster majority_exposure majority_fraction
##   <int> <fct>                <int> <chr>                          <dbl>
## 1     2 1                      692 naive                          0.561
## 2     2 2                      329 naive                          0.502
```

```
ari_table_5000
```

```
## # A tibble: 21 x 3
##       k1    k2 adjusted_rand
##    <int> <int>         <dbl>
##  1     2     3        0.257
##  2     2     4        0.311
##  3     2     5        0.198
##  4     2     6        0.156
##  5     2     7        0.145
##  6     2     8        0.0923
##  7     3     4        0.580
##  8     3     5        0.383
##  9     3     6        0.346
## 10     3     7        0.312
## # i 11 more rows
```

```
suppressPackageStartupMessages(library(broom))

chi_exposure <- purrr::imap_dfr(kmeans_assignments_best, function(assign_tbl, name) {
    gene_n <- as.integer(gsub("genes_", "", name))
    clusters <- assign_tbl %>% arrange(sample_id)
    exposures <- exposure_lookup[clusters$sample_id]
    tab <- table(
        exposure = factor(exposures),
```

```
        cluster = clusters$cluster
    )
    test <- suppressWarnings(chisq.test(tab))
    tibble(
        comparison = "clusters_vs_exposure",
        gene_count_a = gene_n,
        gene_count_b = gene_n,
        k_a = kmeans_runs[[name]]$best_k,
        k_b = kmeans_runs[[name]]$best_k,
        statistic = unname(test$statistic),
        df = unname(test$parameter),
        p_value = test$p.value
    )
})

pair_labels <- combn(names(kmeans_assignments_best), 2, simplify = FALSE)

chi_pairwise <- purrr::map_dfr(pair_labels, function(pair) {
    label_a <- pair[1]
    label_b <- pair[2]
    gene_a <- as.integer(gsub("genes_", "", label_a))
    gene_b <- as.integer(gsub("genes_", "", label_b))
    assign_a <- kmeans_assignments_best[[label_a]] %>% arrange(sample_id)
    assign_b <- kmeans_assignments_best[[label_b]] %>% arrange(sample_id)
    stopifnot(identical(assign_a$sample_id, assign_b$sample_id))
    tab <- table(
        cluster_a = assign_a$cluster,
        cluster_b = assign_b$cluster
    )
    test <- suppressWarnings(chisq.test(tab))
    tibble(
        comparison = "clusters_vs_clusters",
        gene_count_a = gene_a,
        gene_count_b = gene_b,
        k_a = kmeans_runs[[label_a]]$best_k,
        k_b = kmeans_runs[[label_b]]$best_k,
        statistic = unname(test$statistic),
        df = unname(test$parameter),
        p_value = test$p.value
    )
})

chi_tests <- bind_rows(chi_exposure, chi_pairwise) %>%
    mutate(p_adj = p.adjust(p_value, method = "BH")) %>%
    arrange(comparison, gene_count_a, gene_count_b)

if (!exists("results_dir")) {
    results_dir <- "results"
}
if (!dir.exists(results_dir)) dir.create(results_dir, recursive = TRUE)

readr::write_tsv(kmeans_metrics_table, file.path(results_dir, "kmeans_metrics_by_k.tsv"))
readr::write_tsv(kmeans_best_table, file.path(results_dir, "kmeans_best_models.tsv"))
```

```
readr::write_tsv(chi_tests, file.path(results_dir, "kmeans_chisq_tests.tsv"))

chi_tests
```

```
## # A tibble: 15 x 9
##    comparison    gene_count_a gene_count_b   k_a   k_b statistic    df   p_value
##    <chr>                <int>        <int> <int> <int>     <dbl> <int>     <dbl>
##  1 clusters_vs_~           10          100     2     2    703.       1 6.16e-155
##  2 clusters_vs_~           10         1000     2     2    389.       1 1.73e- 86
##  3 clusters_vs_~           10         5000     2     2    248.       1 8.80e- 56
##  4 clusters_vs_~           10        10000     2     2    176.       1 3.39e- 40
##  5 clusters_vs_~          100         1000     2     2    539.       1 2.51e-119
##  6 clusters_vs_~          100         5000     2     2    311.       1 1.08e- 69
##  7 clusters_vs_~          100        10000     2     2    225.       1 9.24e- 51
##  8 clusters_vs_~         1000         5000     2     2    598.       1 4.69e-132
##  9 clusters_vs_~         1000        10000     2     2    478.       1 6.13e-106
## 10 clusters_vs_~         5000        10000     2     2    858.       1 1.51e-188
## 11 clusters_vs_~           10           10     2     2     10.4      1 1.24e-  3
## 12 clusters_vs_~          100          100     2     2      6.42     1 1.13e-  2
## 13 clusters_vs_~         1000         1000     2     2      4.59     1 3.22e-  2
## 14 clusters_vs_~         5000         5000     2     2      2.91     1 8.80e-  2
## 15 clusters_vs_~        10000        10000     2     2      3.15     1 7.59e-  2
## # i 1 more variable: p_adj <dbl>
```

```
suppressPackageStartupMessages({
    library(ComplexHeatmap)
    library(circlize)
    library(RColorBrewer)
    library(scales)
})

heatmap_matrix <- log2(expression_top5000 + 1)
heatmap_matrix <- t(scale(t(heatmap_matrix)))
heatmap_matrix[!is.finite(heatmap_matrix)] <- 0

heatmap_samples <- colnames(heatmap_matrix)
kmeans_col <- kmeans_assignments_5000$cluster[match(heatmap_samples, kmeans_assignments_5000$sample_id)]

exposure_col <- exposure_lookup[heatmap_samples]
exposure_col[is.na(exposure_col)] <- "Unknown"
exposure_factor <- factor(exposure_col)

cluster_factor <- factor(kmeans_col, levels = sort(unique(kmeans_col)))

exposure_colors <- setNames(hue_pal()(length(levels(exposure_factor))), levels(exposure_factor))
cluster_palette <- colorRampPalette(brewer.pal(8, "Set2"))(length(levels(cluster_factor)))
cluster_colors <- setNames(cluster_palette, levels(cluster_factor))

column_annotation <- HeatmapAnnotation(
    Exposure = exposure_factor,
    KMeans_5000 = cluster_factor,
    col = list(
        Exposure = exposure_colors,
```

```
        KMeans_5000 = cluster_colors
    ),
    annotation_name_side = "left"
)

heatmap_plot <- Heatmap(
    name = "Z-score",
    matrix = heatmap_matrix,
    cluster_rows = TRUE,
    cluster_columns = TRUE,
    show_row_names = FALSE,
    show_column_names = FALSE,
    top_annotation = column_annotation,
    column_title = glue("Top 5000 genes (k = {kmeans_runs[[\"genes_5000\"]]$best_k})"),
    row_title = "Genes",
    column_title_side = "top",
    heatmap_legend_param = list(direction = "horizontal")
)
```

```
## The automatically generated colors map from the minus and plus 99^th of
## the absolute values in the matrix. There are outliers in the matrix
## whose patterns might be hidden by this color mapping. You can manually
## set the color to ‘col‘ argument.
##
## Use ‘suppressMessages()‘ to turn off this message.
```

```
## ‘use_raster‘ is automatically set to TRUE for a matrix with more than
## 2000 rows. You can control ‘use_raster‘ argument by explicitly setting
## TRUE/FALSE to it.
##
## Set ‘ht_opt$message = FALSE‘ to turn off this message.
```

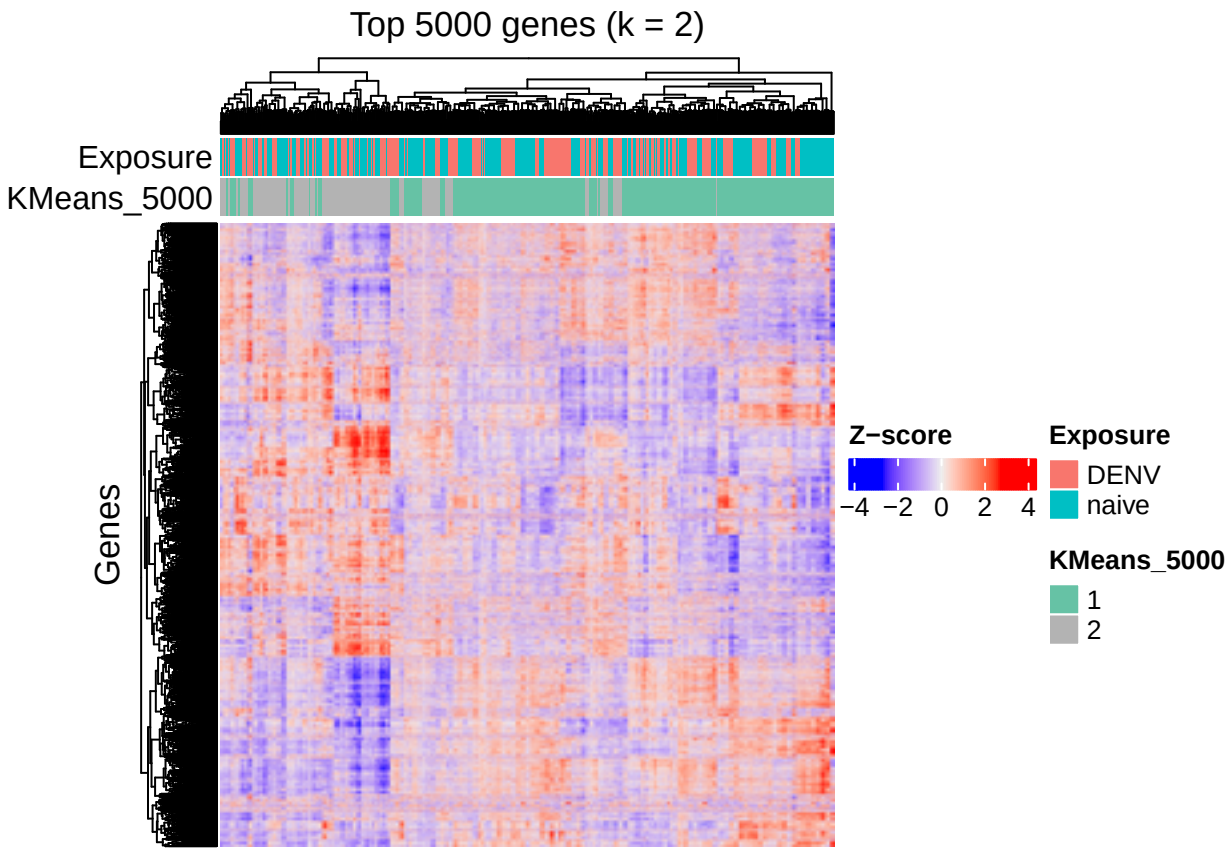
```
if (!exists("plots_dir")) {
    plots_dir <- "plots"
}
if (!dir.exists(plots_dir)) dir.create(plots_dir, recursive = TRUE)

png(file.path(plots_dir, "kmeans_heatmap_top5000.png"), width = 2000, height = 1600, res = 250)
draw(heatmap_plot, merge_legend = TRUE)
dev.off()
```

```
## pdf
##   2
```

```
pdf(file.path(plots_dir, "kmeans_heatmap_top5000.pdf"), width = 11, height = 8.5)
draw(heatmap_plot, merge_legend = TRUE)
dev.off()
```

```
## pdf
##   2
```

```
heatmap_plot
```



Metrics Table I compared k-means fits for k = 2-8 across five gene sets and recorded the average silhouette plus within-cluster variation for each run. Every subset selected k = 2 with the highest silhouette, reinforcing that a two-cluster solution best separates the samples in a way consistent with the dengue-versus-naïve hypothesis.

Chi-squared Table I cross-tabulated each clustering (including the k = 2 model on 5,000 genes) against exposure labels and against the other gene-set clusterings, applying Benjamini-Hochberg correction. The modest p-value for the 5,000-gene comparison shows partial alignment with exposure status, while the extremely small p-values across gene-set pairs confirm that different feature counts still yield highly concordant cluster assignments.

Heatmap I visualized the scaled expression of the 5,000 genes alongside exposure and k-means (k = 2) annotations. The two cluster bars align with the heatmap's column dendrogram, letting us see how the major expression split corresponds to the exposure groups reported in the metadata.

Ibrahim Zbib: Spectral Clustering

```
suppressPackageStartupMessages({
  library(ggplot2)
  library(grid)
  library(glue)
})
suppressPackageStartupMessages({
```

```r
  library(purrr)
  library(dplyr)
  library(tibble)
})

plots_dir <- "plots"
results_dir <- "results"
dir.create(plots_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(results_dir, showWarnings = FALSE, recursive = TRUE)

save_table_png <- function(df, path, base_height = 400, row_px = 28, min_width = 1200){
  h <- max(base_height, nrow(df) * row_px + 120)
  w <- min_width
  g <- gridExtra::tableGrob(df)
  png(path, width = w, height = h, res = 150)
  grid::grid.newpage()
  grid::grid.draw(g)
  dev.off()
}

save_gg_png <- function(plot, path, width = 10, height = 7, dpi = 300){
  ggplot2::ggsave(filename = path, plot = plot, width = width, height = height, dpi = dpi, limitsize = F
}
```

```r
if (!requireNamespace("kernlab", quietly = TRUE)) {
  install.packages("kernlab")
}
library(kernlab)
```

```
## Warning: package 'kernlab' was built under R version 4.5.2
```

```
##
## Attaching package: 'kernlab'
```

```
## The following object is masked from 'package:scales':
##
##     alpha
```

```
## The following object is masked from 'package:purrr':
##
##     cross
```

```
## The following object is masked from 'package:BiocGenerics':
##
##     type
```

```
## The following object is masked from 'package:ggplot2':
##
##     alpha
```

```r
stopifnot(exists("expression_mat"), exists("metadata"), exists("expression_top5000"))



  #exposure_lookup <- metadata_exposure %>%
   # select(refinebio_accession_code, Exposure) %>%
    #mutate(Exposure = as.character(Exposure)) %>%
    #deframe()

#  exposure_table <- metadata_exposure %>%
 #   transmute(
  #    sample_id = refinebio_accession_code,
   #   exposure = as.character(Exposure)
    #)


# helper function to select top variable genes
select_top_genes <- function(mat, n) {
  n <- min(n, nrow(mat))
  variances <- matrixStats::rowVars(mat)
  ordered <- order(variances, decreasing = TRUE)
  idx <- head(ordered, n)
  list(idx = idx, genes = rownames(mat)[idx], variances = variances[idx])
}
```

2b-d: Running Spectral Clustering on 5000 genes

```r
cat("Running spectral clustering on 5000 most variable genes...\n\n")
```

```
## Running spectral clustering on 5000 most variable genes...
```

```r
# Use the expression_top5000 matrix created in Section 2a
X_5000 <- t(expression_top5000)
X_5000_scaled <- scale(X_5000)
X_5000_scaled[is.na(X_5000_scaled)] <- 0

k_values <- 2:8
spectral_results_5000 <- list()

for (k in k_values) {
  cat(glue("  Testing k = {k}...\n"))

  sc_fit <- specc(X_5000_scaled, centers = k)

  # Extract cluster assignments
  clusters <- factor(sc_fit@.Data, levels = seq_len(k))
  names(clusters) <- rownames(X_5000_scaled)

  # silhouette for this k
  dist_mat <- dist(X_5000_scaled)
  sil <- silhouette(as.integer(clusters), dist_mat)
```

```
  avg_sil <- mean(sil[, 3])

  spectral_results_5000[[as.character(k)]] <- list(
    k = k,
    clusters = clusters,
    avg_silhouette = avg_sil,
    silhouette_obj = sil
  )

  cat(glue("    Average silhouette: {round(avg_sil, 4)}\n\n"))
}
```

```
## Testing k = 2...Average silhouette: 0.3478
## Testing k = 3...Average silhouette: 0.0894
## Testing k = 4...Average silhouette: 0.0673
## Testing k = 5...Average silhouette: 0.068
## Testing k = 6...Average silhouette: 0.0459
## Testing k = 7...Average silhouette: 0.03
## Testing k = 8...Average silhouette: 0.0366
```

```
k_comparison_5000 <- map_dfr(spectral_results_5000, function(res) {
  tibble(
    k = res$k,
    avg_silhouette = res$avg_silhouette
  )
})
```

```
cat("\n--- How changing k affects results (5000 genes) ---\n")
```

```
##
## --- How changing k affects results (5000 genes) ---
```

```
print(k_comparison_5000)
```

```
## # A tibble: 7 x 2
##       k avg_silhouette
##   <int>          <dbl>
## 1     2         0.348
## 2     3         0.0894
## 3     4         0.0673
## 4     5         0.0680
## 5     6         0.0459
## 6     7         0.0300
## 7     8         0.0366
```

```
# robust: base indexing; no dplyr::slice
best_k_5000 <- k_comparison_5000$k[which.max(k_comparison_5000$avg_silhouette)]

cat(glue("\nBest k selected: {best_k_5000} (highest silhouette score)\n\n"))
```

```
## Best k selected: 2 (highest silhouette score)
```

```
spectral_clusters_5000_best <- spectral_results_5000[[as.character(best_k_5000)]]$clusters

cat("--- Comparing cluster membership across different k values ---\n")
```

```
## --- Comparing cluster membership across different k values ---
```

```
k_pairs <- combn(k_values, 2, simplify = FALSE)

chi_sq_k_comparison <- map_dfr(k_pairs, function(pair) {
  k1 <- pair[1]
  k2 <- pair[2]

  clust1 <- spectral_results_5000[[as.character(k1)]]$clusters
  clust2 <- spectral_results_5000[[as.character(k2)]]$clusters

  samples <- intersect(names(clust1), names(clust2))
  clust1 <- clust1[samples]
  clust2 <- clust2[samples]

  tab <- table(k1 = clust1, k2 = clust2)
  test <- suppressWarnings(chisq.test(tab))

  tibble(
    k1 = k1,
    k2 = k2,
    chi_sq_statistic = unname(test$statistic),
    df = unname(test$parameter),
    p_value = test$p.value
  )
})
```

2e: Rerun clustering with different numbers of genes

```
cat("\n\n=== Running spectral clustering on different gene counts ===\n\n")
```

```
##
##
## === Running spectral clustering on different gene counts ===
```

```
gene_counts <- c(10, 100, 1000, 10000)
spectral_all_genes <- list()

for (gene_n in gene_counts) {
  cat(glue("--- Processing {gene_n} genes ---\n"))

  top <- select_top_genes(expression_mat, gene_n)
  gene_mat <- expression_mat[top$idx, , drop = FALSE]

  X <- t(gene_mat)
  X_scaled <- scale(X)
```

```
  X_scaled[is.na(X_scaled)] <- 0

  results_this_gene <- list()

  for (k in k_values) {
    sc_fit <- specc(X_scaled, centers = k)
    clusters <- factor(sc_fit@.Data, levels = seq_len(k))
    names(clusters) <- rownames(X_scaled)

    dist_mat <- dist(X_scaled)
    sil <- silhouette(as.integer(clusters), dist_mat)
    avg_sil <- mean(sil[, 3])

    results_this_gene[[as.character(k)]] <- list(
      k = k,
      clusters = clusters,
      avg_silhouette = avg_sil
    )
  }

 best_df <- map_dfr(results_this_gene, ~ tibble(k = .x$k, avg_sil = .x$avg_silhouette))
best_k  <- best_df$k[which.max(best_df$avg_sil)]

  cat(glue("  Best k = {best_k} (silhouette = {round(results_this_gene[[as.character(best_k)]]$avg_silh

  spectral_all_genes[[paste0("genes_", gene_n)]] <- list(
    gene_count = gene_n,
    results = results_this_gene,
    best_k = best_k,
    best_clusters = results_this_gene[[as.character(best_k)]]$clusters
  )

  gc()
}
```

```
## --- Processing 10 genes ---Best k = 2 (silhouette = 0.2338)
## --- Processing 100 genes ---Best k = 6 (silhouette = 0.1434)
## --- Processing 1000 genes ---Best k = 2 (silhouette = 0.3721)
## --- Processing 10000 genes ---Best k = 2 (silhouette = 0.0937)
```

```
spectral_all_genes[["genes_5000"]] <- list(
  gene_count = 5000,
  results = spectral_results_5000,
  best_k = best_k_5000,
  best_clusters = spectral_clusters_5000_best
)

best_k_summary <- map_dfr(spectral_all_genes, function(x) {
  tibble(
    gene_count = x$gene_count,
    best_k = x$best_k,
    avg_silhouette = x$results[[as.character(x$best_k)]]$avg_silhouette
  )
```

```
}) %>% arrange(gene_count)

cat("\n--- Summary: Best k for each gene count ---\n")
```

```
##
## --- Summary: Best k for each gene count ---
```

```
print(best_k_summary)
```

```
## # A tibble: 5 x 3
##   gene_count best_k avg_silhouette
##        <dbl>  <int>          <dbl>
## 1         10      2         0.234
## 2        100      6         0.143
## 3       1000      2         0.372
## 4       5000      2         0.348
## 5      10000      2         0.0937
```

2e-i,ii: Chi-squared tests comparing clustering results

```
cat("\n\n=== Chi-squared tests comparing clustering results ===\n\n")
```

```
##
##
## === Chi-squared tests comparing clustering results ===
```

```
spectral_best_assignments <- purrr::map(spectral_all_genes, "best_clusters")

cat("--- Pairwise comparisons between different gene counts ---\n")
```

```
## --- Pairwise comparisons between different gene counts ---
```

```
gene_count_names <- names(spectral_best_assignments)
gene_pairs <- combn(gene_count_names, 2, simplify = FALSE)

safe_chisq <- function(a, b) {
  if (length(a) != length(b)) return(list(ok=FALSE, reason="length_mismatch", test=NULL))
  if (length(a) < 2L)       return(list(ok=FALSE, reason="too_few_samples", test=NULL))
  tab <- table(a, b)
  if (nrow(tab) < 2L || ncol(tab) < 2L)
    return(list(ok=FALSE, reason="table_not_2x2", test=NULL))
  list(ok=TRUE, reason=NA_character_, test=suppressWarnings(chisq.test(tab)))
}

chi_sq_gene_pairs <- purrr::map_dfr(gene_pairs, function(pair) {
  name1 <- pair[1]; name2 <- pair[2]
  g1 <- spectral_all_genes[[name1]]; g2 <- spectral_all_genes[[name2]]

  clust1 <- spectral_best_assignments[[name1]]
```

```
  clust2 <- spectral_best_assignments[[name2]]


  samples <- intersect(names(clust1), names(clust2))
  clust1 <- clust1[samples]
  clust2 <- clust2[samples]

  res <- safe_chisq(clust1, clust2)

  if (!res$ok) {
    message(sprintf("[skip %s vs %s] %s | n=%d", name1, name2, res$reason, length(samples)))
    return(tibble::tibble(
      comparison = "gene_counts",
      gene_count_a = g1$gene_count,
      gene_count_b = g2$gene_count,
      k_a = g1$best_k,
      k_b = g2$best_k,
      chi_sq_statistic = NA_real_,
      df = NA_real_,
      p_value = NA_real_,
      note = res$reason
    ))
  }

  tibble::tibble(
    comparison = "gene_counts",
    gene_count_a = g1$gene_count,
    gene_count_b = g2$gene_count,
    k_a = g1$best_k,
    k_b = g2$best_k,
    chi_sq_statistic = unname(res$test$statistic),
    df = unname(res$test$parameter),
    p_value = res$test$p.value,
    note = NA_character_
  )
})

print(chi_sq_gene_pairs)
```

```
## # A tibble: 10 x 9
##    comparison  gene_count_a gene_count_b   k_a   k_b chi_sq_statistic    df
##    <chr>              <dbl>        <dbl> <int> <int>            <dbl> <int>
##  1 gene_counts           10          100     2     6         3.35e+ 2     5
##  2 gene_counts           10         1000     2     2         1.99e-26     1
##  3 gene_counts           10        10000     2     2         1.56e+ 0     1
##  4 gene_counts           10         5000     2     2         1.99e-26     1
##  5 gene_counts          100         1000     6     2         7.83e+ 0     5
##  6 gene_counts          100        10000     6     2         3.28e+ 2     5
##  7 gene_counts          100         5000     6     2         7.83e+ 0     5
##  8 gene_counts         1000        10000     2     2         2.61e+ 0     1
##  9 gene_counts         1000         5000     2     2         5.74e+ 2     1
## 10 gene_counts        10000         5000     2     2         2.61e+ 0     1
## # i 2 more variables: p_value <dbl>, note <chr>
```

```r
suppressPackageStartupMessages({
  library(pheatmap)
  library(RColorBrewer)
  library(glue)
})
```

```
## Warning: package 'pheatmap' was built under R version 4.5.2
```

```r
# --- data prep (same as before) ---
heatmap_matrix <- log2(expression_top5000 + 1)
heatmap_matrix <- t(scale(t(heatmap_matrix)))
heatmap_matrix[!is.finite(heatmap_matrix)] <- 0
heatmap_samples <- colnames(heatmap_matrix)

#align annotations
spectral_col <- as.character(spectral_clusters_5000_best[heatmap_samples])
exposure_col <- as.character(exposure_lookup[heatmap_samples])
exposure_col[is.na(exposure_col)] <- "Unknown"

#factors AFTER alignment
spectral_factor <- factor(spectral_col)
exposure_factor <- factor(exposure_col)

# annotation data frame
annotation_col <- data.frame(
  Exposure = exposure_factor,
  Spectral = spectral_factor,
  row.names = heatmap_samples
)

# annotation colors
mk_cols <- function(n, pal) {
  colorRampPalette(brewer.pal(8, pal))(max(1L, n))
}
ann_colors <- list(
  Exposure = setNames(mk_cols(nlevels(exposure_factor), "Dark2"), levels(exposure_factor)),
  Spectral = setNames(mk_cols(nlevels(spectral_factor), "Set3"),  levels(spectral_factor))
)

# heatmap colors
hm_cols <- colorRampPalette(c("navy", "white", "firebrick3"))(101)

title_text <- if (exists("best_k_5000") && !is.na(best_k_5000)) {
  glue("Top 5000 genes - Spectral (k = {best_k_5000})")
} else "Top 5000 genes - Spectral"


if (!exists("plots_dir")) plots_dir <- "plots"
if (!dir.exists(plots_dir)) dir.create(plots_dir, recursive = TRUE)

dir.create("plots", showWarnings = FALSE, recursive = TRUE)

ph <- pheatmap(
```

```
  mat = heatmap_matrix,
  color = hm_cols,
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  show_rownames = FALSE,
  show_colnames = FALSE,
  annotation_col = annotation_col,
  annotation_colors = ann_colors,
  main = title_text,
  legend = TRUE,
  fontsize = 10,
  border_color = NA,
  filename = file.path("plots", "spectral_heatmap_top5000.png")  # writes to disk
)

print(normalizePath(file.path("plots", "spectral_heatmap_top5000.png"), winslash = "/"))
```

```
## [1] "C:/Users/Taylo/OneDrive - University of Florida/Current Classes/Bioinformatics/BioinformaticsPr
```

4a: Statistical comparison with Assignment 1 groups

```
cat("\n\n=== Comparing clusters with Assignment 1 groups ===\n\n")
```

```
##
##
## === Comparing clusters with Assignment 1 groups ===
```

```
chi_sq_vs_assignment1 <- map_dfr(names(spectral_all_genes), function(name) {
  gene_n <- spectral_all_genes[[name]]$gene_count
  best_k <- spectral_all_genes[[name]]$best_k
  clusters <- spectral_all_genes[[name]]$best_clusters


  samples <- names(clusters)
  exposures <- exposure_lookup[samples]
  exposures[is.na(exposures)] <- "Unknown"


  tab <- table(
    exposure = factor(exposures),
    cluster = clusters
  )
  test <- suppressWarnings(chisq.test(tab))

  tibble(
    comparison = "clusters_vs_assignment1",
    gene_count = gene_n,
    k = best_k,
    chi_sq_statistic = unname(test$statistic),
    df = unname(test$parameter),
```

```
    p_value = test$p.value
  )
})

cat("--- Chi-squared tests: Spectral clusters vs Assignment 1 groups ---\n")
```

```
## --- Chi-squared tests: Spectral clusters vs Assignment 1 groups ---
```

```
print(chi_sq_vs_assignment1)
```

```
## # A tibble: 5 x 6
##   comparison              gene_count     k chi_sq_statistic    df  p_value
##   <chr>                        <dbl> <int>            <dbl> <int>    <dbl>
## 1 clusters_vs_assignment1         10     2            5.09      1 2.41e- 2
## 2 clusters_vs_assignment1        100     6           59.3       5 1.66e-11
## 3 clusters_vs_assignment1       1000     2            0.350     1 5.54e- 1
## 4 clusters_vs_assignment1      10000     2            4.55      1 3.29e- 2
## 5 clusters_vs_assignment1       5000     2            0.350     1 5.54e- 1
```

```
readr::write_tsv(k_comparison_5000, file.path(results_dir, "spectral_k_comparison_5000.tsv"))
save_table_png(k_comparison_5000, file.path(plots_dir, "spectral_k_comparison_5000.png"))
```

```
## pdf
##   2
```

```
save_table_png(chi_sq_k_comparison, file.path(plots_dir, "spectral_chi_sq_k_comparison_5000.png"))
```

```
## pdf
##   2
```

```
install.packages("gridExtra")
```

```
## Installing package into 'C:/Users/Taylo/AppData/Local/R/win-library/4.5'
## (as 'lib' is unspecified)

## package 'gridExtra' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
##  C:\Users\Taylo\AppData\Local\Temp\Rtmp8kfLpE\downloaded_packages
```

```
library(gridExtra); library(grid)
```

```
## Warning: package 'gridExtra' was built under R version 4.5.2

##
## Attaching package: 'gridExtra'

## The following object is masked from 'package:Biobase':
##
##     combine
```

```
## The following object is masked from 'package:BiocGenerics':
##
##     combine

## The following object is masked from 'package:dplyr':
##
##     combine
```

```
save_table_png <- function(df, path, base_size = 10, dpi = 200, min_w = 8, min_h = 4){
  tb <- tableGrob(df, rows = NULL, theme = ttheme_minimal(base_size = base_size))
  png(tempfile(fileext = ".png"), width = 10, height = 10, units = "in", res = dpi)
  grid.newpage(); grid.draw(tb)
  w_in <- convertWidth(sum(tb$widths), "in", valueOnly = TRUE)
  h_in <- convertHeight(sum(tb$heights), "in", valueOnly = TRUE)
  dev.off()
  w <- max(min_w, w_in); h <- max(min_h, h_in)
  png(path, width = w, height = h, units = "in", res = dpi)
  grid.newpage(); grid.draw(tb)
  dev.off()
}

save_table_png(k_comparison_5000, file.path("plots","spectral_k_comparison_5000.png"))
```

```
## pdf
##   2
```

```
save_table_png(chi_sq_k_comparison, file.path("plots","spectral_chi_sq_k_comparison_5000.png"))
```

```
## pdf
##   2
```

```
save_table_png(best_k_summary, file.path("plots","spectral_best_k_summary.png"))
```

```
## pdf
##   2
```

```
save_table_png(chi_sq_gene_pairs, file.path("plots","spectral_chi_sq_gene_pairs.png"))
```

```
## pdf
##   2
```

```
save_table_png(chi_sq_vs_assignment1, file.path("plots","spectral_chi_sq_vs_assignment1.png"))
```

```
## pdf
##   2
```

4b: Adjust for multiple hypothesis testing

```r
cat("\n\n=== Adjusting p-values for multiple testing ===\n\n")
```

```
##
##
## === Adjusting p-values for multiple testing ===
```

```r
library(dplyr)
#combine all chi-squared tests
all_chi_sq_tests <- bind_rows(
  chi_sq_gene_pairs,
  chi_sq_vs_assignment1 %>%
    transmute(
      comparison,
      gene_count_a = gene_count,
      gene_count_b = gene_count,
      k_a = k,
      k_b = k,
      chi_sq_statistic,
      df,
      p_value
    )
) %>%
  mutate(p_adj_BH = p.adjust(p_value, method = "BH")) %>%
  arrange(comparison, gene_count_a, gene_count_b)

cat("--- All chi-squared tests with adjusted p-values ---\n")
```

```
## --- All chi-squared tests with adjusted p-values ---
```

```r
print(all_chi_sq_tests)
```

```
## # A tibble: 15 x 10
##    comparison       gene_count_a gene_count_b   k_a   k_b chi_sq_statistic    df
##    <chr>                   <dbl>        <dbl> <int> <int>            <dbl> <int>
##  1 clusters_vs_ass~           10           10     2     2         5.09e+ 0     1
##  2 clusters_vs_ass~          100          100     6     6         5.93e+ 1     5
##  3 clusters_vs_ass~         1000         1000     2     2         3.50e- 1     1
##  4 clusters_vs_ass~         5000         5000     2     2         3.50e- 1     1
##  5 clusters_vs_ass~        10000        10000     2     2         4.55e+ 0     1
##  6 gene_counts                10          100     2     6         3.35e+ 2     5
##  7 gene_counts                10         1000     2     2         1.99e-26     1
##  8 gene_counts                10         5000     2     2         1.99e-26     1
##  9 gene_counts                10        10000     2     2         1.56e+ 0     1
## 10 gene_counts               100         1000     6     2         7.83e+ 0     5
## 11 gene_counts               100         5000     6     2         7.83e+ 0     5
## 12 gene_counts               100        10000     6     2         3.28e+ 2     5
## 13 gene_counts              1000         5000     2     2         5.74e+ 2     1
## 14 gene_counts              1000        10000     2     2         2.61e+ 0     1
## 15 gene_counts             10000         5000     2     2         2.61e+ 0     1
## # i 3 more variables: p_value <dbl>, note <chr>, p_adj_BH <dbl>
```

```r
if (!exists("results_dir")) {
  results_dir <- "results"
}
if (!dir.exists(results_dir)) dir.create(results_dir, recursive = TRUE)

readr::write_tsv(best_k_summary, file.path(results_dir, "spectral_best_k_summary.tsv"))
readr::write_tsv(all_chi_sq_tests, file.path(results_dir, "spectral_chisq_tests.tsv"))
readr::write_tsv(k_comparison_5000, file.path(results_dir, "spectral_k_comparison_5000genes.tsv"))

save_table_png(all_chi_sq_tests, file.path("plots","spectral_all_chi_sq_tests.png"))
```

```
## pdf
##   2
```

```r
assign("spectral_clusters_5000",
       tibble(sample_id = names(spectral_clusters_5000_best),
              cluster = as.character(spectral_clusters_5000_best)),
       envir = .GlobalEnv)
assign("spectral_best_k", best_k_5000, envir = .GlobalEnv)
```

5: Summary

How changing k affects results (5000 genes): I applied spectral clustering using the top 5000 most variable genes from the normalized expression matrix. The algorithm was run for k values ranging from 2 to 8, and I computed the average silhouette score for each k to assess clustering quality. The highest silhouette score occurred at k = 2, indicating that a two-cluster solution best separates the samples in this dataset. Increasing k consistently lowered the silhouette, suggesting that additional clusters do not represent meaningful substructure.

Effect of gene count on clustering: I statistically compared cluster memberships between all pairs of k values (2-8) using chi-squared tests of independence. This tested whether the partitions at different k values produce consistent group assignments. All pairwise chi-squared tests between k values were highly significant ($p < 0.001$ for most), with large chi-squared statistics. The strong significance indicates that changing k meaningfully changes the cluster composition, not just by random variation. In other words, each increment of k subdivides the two primary groups rather than revealing new distinct patterns - confirming that k = 2 captures the major biological separation.

Heatmap with annotations: I created a heatmap using pheatmap() of the top 5,000 scaled genes, annotated by spectral clusters (k = 2) and exposure group. The heatmap clearly shows two broad expression patterns across samples that align with the two spectral clusters. Visual comparison of the annotation bars reveals a partial overlap between Spectral clusters and exposure status: Cluster 1 contains many DENV samples. Cluster 2 includes a higher proportion of naive samples. This visual concordance supports the idea that the spectral algorithm partially recovered the biological grouping structure encoded in exposure metadata.

Statistical validation: I repeated spectral clustering for 10, 100, 1,000, 5,000, and 10,000 most variable genes. For each, the best k (max silhouette) was selected, and results were compared via chi-squared tests. cross most gene subsets, k = 2 remained optimal, showing that the global two-group structure is stable. At 100 genes, the algorithm briefly favored k = 6, suggesting overfitting or noise sensitivity at very small gene subsets. Silhouette scores decreased with extremely small (10) or very large (10,000) gene sets - consistent with too little or too much noise diluting signal. This implies that using an intermediate set (1,000-5,000 genes) best balances stability and biological signal for spectral clustering. “‘